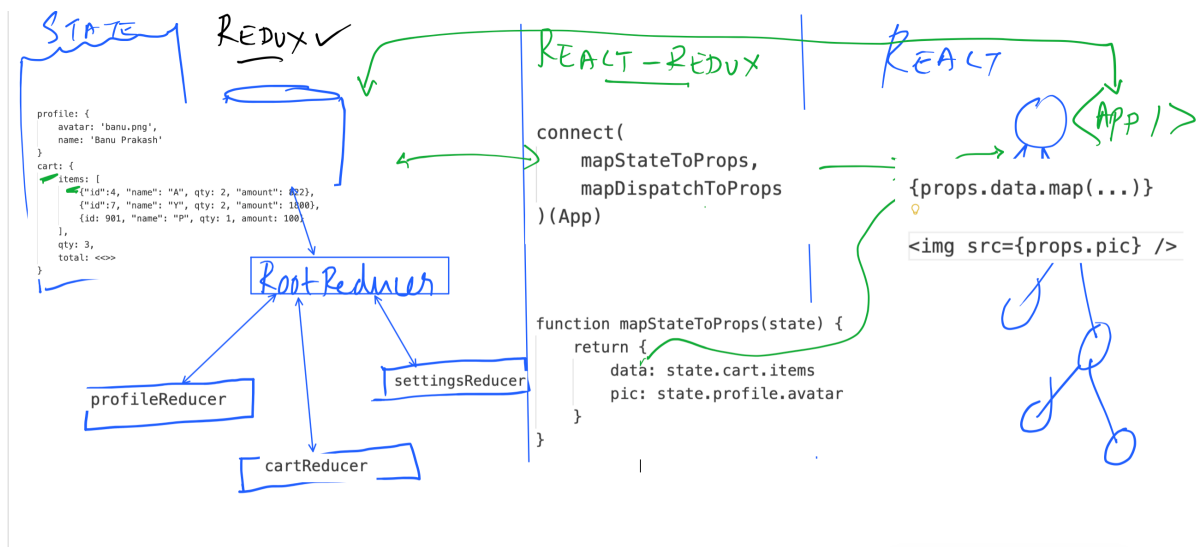
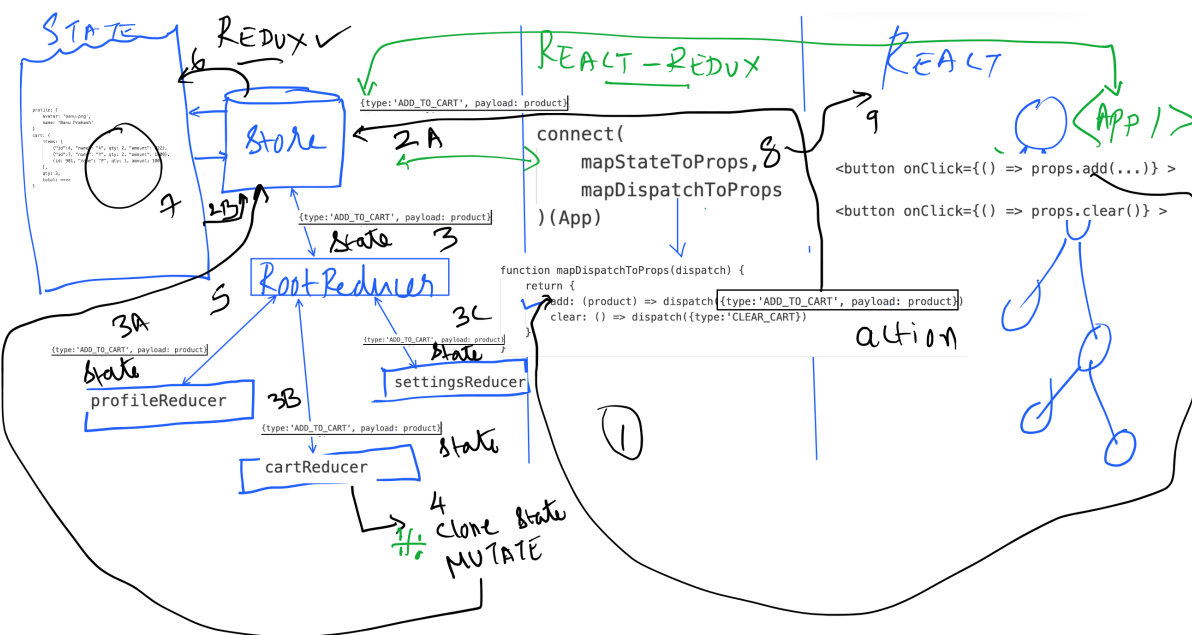
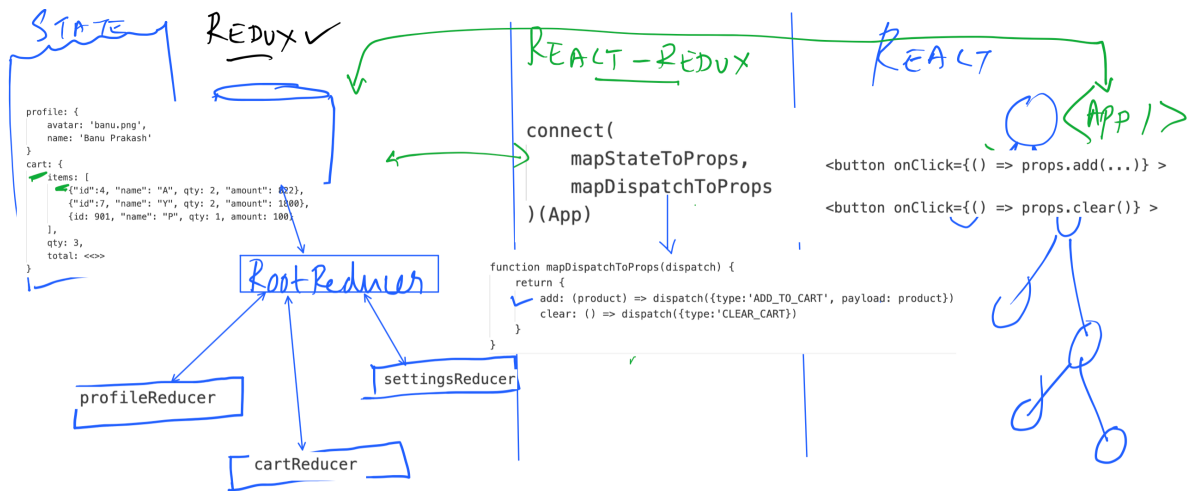


Context Vs REDUX

Key Differences

Constraint 	Context API	Redux
Type	Built-into React	External library (requires installation)
Setup	Minimal and easy	More extensive initially, but simplified with Redux Toolkit
Use Case	Simple, low-frequency updates (e.g., UI themes)	Complex, high-frequency updates and large-scale applications
Performance	Can cause all consuming components to re-render if any part of the context value changes	Highly performant with granular updates, re-rendering only components that use the specific changed state via selectors
Debugging	Difficult to trace state changes over time	Excellent with robust Redux DevTools (time-travel debugging, action history)
Features	Primarily for data passing (dependency injection)	Offers middleware (for side effects like API calls), state persistence, and more
Flexibility	Limited to React applications	UI-agnostic; can be used with React, Angular, Vue, or vanilla JS





```

const initialState = {
  contacts: []
}

const contactSlice = createSlice({
  name: 'contacts',
  initialState,
  reducers: {
    addContact: (state, action) => {
      state.contacts.push(action.payload); // internally it's a clone
    },
    removeContact: (state, action) => {
      let newData = state.contacts.filter(contact => contact.email !== action.payload)
      state.contacts = newData;
    },
    clearContacts: (state) => {
      state.contacts = [];
    }
  }
})

export const store = configureStore({
  // root reducer
  reducer: {
    contacts: contactReducer,
    profile: profileReducer
  }
})

const initialState = {
  "avatar": "banu.png",
  "displayName": "Banu Prakash"
}

const profileSlice = createSlice({
  name: 'profile',
  initialState,
  reducers: {
  }
})

let {avatar} = useSelector(state => state.profile);

```

```

const initialState = {
  contacts: []
}

const contactSlice = createSlice({
  name: 'contacts',
  initialState,
  reducers: {
    addContact: (state, action) => {
      state.contacts.push(action.payload); // internally it's a clone
    },
    removeContact: (state, action) => {
      let newData = state.contacts.filter(contact => contact.email !== action.payload)
      state.contacts = newData;
    },
    clearContacts: (state) => {
      state.contacts = [];
    },
  },
});

```

```

export const store = configureStore({
  // root reducer
  reducer: {
    contacts: contactReducer,
    profile: profileReducer
  }
});

```

`let {contacts} = useSelector(state => state.contacts);`

```

const initialState = {
  contacts: []
}

const contactSlice = createSlice({
  name: 'contacts',
  initialState,
  reducers: {
    addContact: (state, action) => {
      state.contacts.push(action.payload); // internally it's a clone
    },
    removeContact: (state, action) => {
      let newData = state.contacts.filter(contact => contact.email !== action.payload)
      state.contacts = newData;
    },
    clearContacts: (state) => {
      state.contacts = [];
    },
  },
});

```

```

export const store = configureStore({
  // root reducer
  reducer: {
    contacts: contactReducer,
    profile: profileReducer
  }
});

```

CLONE

```

{
  contacts: {
    contacts: []
  },
  profile: {
    avatar: 'banu.png',
    displayName: 'Banu Prakash'
  }
}

```

```

function doSubmit() {
  let contact = {
    email: emailRef.current.value,
    name: nameRef.current.value
  }
  dispatch(addContact(contact));
}

```